

Project 2: Schema Linking with Supervised Fine-Tuning

1 Learning Outcomes

By completing this project, you will:

1. Build an end-to-end SFT pipeline for a small language model (SLM): data formatting, prompt design, parameter-efficient training (LoRA/QLoRA), and structured-output decoding.
2. Develop training data for a structured-output task, with attention to curation, schema serialization, and augmentation.
3. Apply set-based identifier-level evaluation metrics (precision, recall, F1) for schema linking, and optionally design your own diagnostic metrics.
4. Use RapidFire AI OSS to run multi-config SFT experiments and converge on better outcomes through disciplined experimentation.
5. Develop engineering judgment about SLM capacity, training-data scale, and prompt-context tradeoffs for real-world structured-prediction tasks.

2 High-Level Description

You will build an SLM-based *schema linking* system, fine-tuned with SFT, and use RapidFire AI's multi-config experimentation framework to explore and optimize your training pipeline.

The task. Schema linking is a core sub-task of natural-language-to-SQL: given an NL question and a database schema, identify the tables and columns the underlying SQL would reference. It is also useful in its own right for query routing, retrieval over relational sources, and grounding LLM agents. Your system must produce structured JSON naming the linked tables and columns:

```
{"TableName1": ["Col1", "Col2"], "TableName2": ["Col3"]}
```

The dataset. Built from **SNAILS** [1], a UCSD ADALab artifact suite of real-world databases with 503 hand-labeled NL question / gold SQL pairs:

- **Domains:** national-parks biodiversity, NTSB crash statistics, NY state education data, and SAP business resource planning.
- **17 db_id values:** SAP SBOD is segmented into 9 business-area schema modules so each fits comfortably into an SLM prompt; combined with 8 standalone databases.
- **Splits:** we have extracted ground-truth schema-link annotations from each gold SQL and released train/validation splits. The hidden test set is drawn from the same source distribution (see Section 4 for the important note on test set composition).

Deployment constraint. Your model must be *locally runnable* on a single GPU within DSMLP (Section 3). No proprietary API calls at inference time. The pedagogical point is to expose you to deploying small, controllable, on-prem models for production grounding tasks.

Why SFT a small model instead of few-shotting a frontier API? A strong few-shot prompt to GPT or Claude would also produce reasonable schema links. The motivation for SFT'ing an SLM

here is not raw accuracy on this benchmark—it is what makes small specialists the right choice for production grounding sub-modules:

- **Cost and latency at scale:** schema linking is called on every NL query in an NL-to-SQL stack; a LoRA'd $\leq 2\text{B}$ model on local hardware is $100\text{--}1000\times$ cheaper per call with bounded latency.
- **Output discipline:** an SFT'd model emits the exact JSON contract reliably because that contract *is* its training distribution.
- **Identifier vocabulary discipline:** training only on schema-grounded extraction keeps the model inside the provided schema's identifiers, directly improving precision.
- **Data residency:** many real deployments cannot send schemas or queries to external APIs. The schema *is* the IP.

The goal is engineering judgment for when and how to build such specialists, not to claim that a 2B SFT'd model beats Opus on a 100-question benchmark.

Three components.

1. **Training Data Strategy:** assemble, format, and serialize your training examples. The released split is your starting point, not your finishing line.
2. **System Building:** SFT an SLM that generates a valid `schema_links` JSON.
3. **Experimentation and Evaluation:** use RapidFire AI OSS to systematically compare multiple SFT configurations and analyze results.

3 Infrastructure

- **Compute:** SDSC DSMLP, accessible at <https://datahub.ucsd.edu/>. Each team receives a MIG slice of an NVIDIA RTX PRO 6000 Blackwell with **$\sim 24\text{ GB GPU memory}$** . Your `main.py` should run inference out of the box on this slice for whatever batch size you specify. Launch instructions on Piazza.
- **Hard constraint: base model must be $\leq 2\text{B}$ parameters.** This cap reflects the production reality the project is teaching: schema linking sits in the inner loop of NL-to-SQL stacks, so per-query inference latency and serving cost dominate—a sub-2B specialist runs in tens of milliseconds on commodity GPUs and is $5\text{--}50\times$ cheaper to serve at scale than 8B+ alternatives. The cap also ensures SFT does real work on this project: at $\leq 2\text{B}$ the base is genuinely weak zero-shot, so your fine-tuning meaningfully moves the needle, and full FT vs. LoRA vs. QLoRA become tractable comparisons on the 24 GB slice.
- **Experimentation Framework:** You can use *rapidfireai* again, with its `RFGGridSearch` or `RFRandomSearch` to launch and compare configs concurrently. Use the IC Ops (Stop, Resume, Clone-Modify) to redirect compute from underperforming configs to promising ones.
- **SFT API:** RapidFire AI wraps HuggingFace TRL's `SFTTrainer` via `RFSFTConfig`, covering dataset loading, tokenization, LoRA/PEFT adapter setup, and training-loop instrumentation. Work through the RapidFire AI SFT walkthrough and tutorial Colab before starting.
- **Base Models (suggested, all $\leq 2\text{B}$ and instruction-tuned):** Qwen2.5-0.5B/1.5B-Instruct, Qwen3-0.6B/1.7B (set `enable_thinking=False` in the chat template so the model emits direct structured output without a CoT preamble), Llama-3.2-1B-Instruct, SmolLM2-1.7B-Instruct.

All have mature TRL + PEFT support and ChatML/Llama chat templates. Picking the right base model and size within this cap is itself a meaningful design choice—families differ in pretraining mix, tokenizer, and structured-output behavior even at similar scales.

- **No external inference APIs:** your final `main.py` must run locally on DSMLP and must not depend on OpenAI/Anthropic/Google APIs at inference. You may use such APIs *offline* for training data augmentation if you document it.

GPU Budget: 30–60 GPU-hours per team—designed to be sufficient for an excellent submission, including the 8+ configs required for the report. You are not expected or required to use compute beyond DSMLP. Idle pods burn the budget for no learning gain.

Training Knobs You May Vary

- **Base model:** family and size within the $\leq 2\text{B}$ cap (e.g., Qwen2.5-0.5B vs. -1.5B; Qwen vs. Llama vs. SmolLM).
- **Adapter:** full fine-tune vs. LoRA vs. QLoRA; LoRA rank, alpha, dropout, target modules.
- **Optimization:** learning rate, schedule, batch size, gradient accumulation, epochs, sequence length.
- **Prompt template:** schema serialization (full dump, PK/FK-annotated, type-annotated, with sample values) and question formatting.
- **Output format:** raw JSON, JSON Lines, custom delimiter format; with or without “thinking” tokens; with or without a candidate-tables list.
- **Training data:** released split alone, with paraphrases, with synthetic NL-SQL pairs, with SNAILS naturalness-modified identifier variants, etc.

Pick a handful you believe will matter most and study them systematically.

4 Release Packet Content

See `release_packet/README.md` for exact formats and example commands.

1. **Database Schemas:** folder `schemas/` with one Spider-format JSON per `db_id` (17 files: table names, column names, types, PKs, FKs). `_index.json` summarizes the collection.
2. **Training Set:** `train.json`, 301 examples with `question_id`, `db_id`, `question`, `gold_sql`, `schema_links`.
3. **Released Validation Set:** `validation.json`, 101 examples in the same format. The parallel files `validation_input.json` and `validation_gold_schema_links.json` match the I/O contract for grading.
4. **Evaluation Script:** `eval.py`, computing set-based table- and column-level P, R, F1. Same script used for hidden-test grading.
5. **SQL-to-Schema-Links Helper:** `sql_to_schema_links.py`, the parser used to extract ground truth from SNAILS gold SQL. Use it to label any additional SQL you write or synthesize.
6. **Sample Inference Stub:** `sample_main.py`, documenting the CLI and I/O contract your `main.py` must follow.
7. **Hidden Test Set:** held-out set over the same 17 schemas, scored with `eval.py`. *Important:* the hidden test set is **not** simply the SNAILS examples we did not release—it may include

questions we authored ourselves, paraphrased variants, and identifier-naturalness variants. Reverse-engineering the test set from public SNAILS artifacts (e.g., training on unreleased SNAILS gold SQL) is both an integrity violation *and* an unproductive strategy.

8. **Evaluation Metrics:** for each question we compute precision, recall, and F1 over two identifier sets:

- *Table identifiers:* the set of table names referenced.
- *Column identifiers:* the set of `table.column` pairs referenced.

Per-question P/R/F1 are macro-averaged across the test set to produce $P_T, R_T, F1_T$ and $P_C, R_C, F1_C$. Each macro-averaged value is the mean of per-question scores, NOT computed from the macro-averaged P and R.

Important details:

- Identifier comparisons are case-insensitive.
 - Identifiers in your prediction not present in the target schema count as false positives (lowering precision) and are also reported as hallucination counts for debugging.
 - *Wildcard boundary:* SQL wildcards do not link specific columns. `select count(*) from t` yields `{"t": []}`; `select * from t where x=1` yields `{"t": ["x"]}`. You must emit the table with the correct (possibly empty) column list—omitting the table is a recall miss.
 - *Empty-gold boundary:* gold sets are empty only at the column level for wildcard queries like `select count(*) from t`. When this happens: if your prediction is also empty, $P = R = F1 = 1$ for that metric; if your prediction is non-empty, $P = 0, R = 1$ (vacuously), $F1 = 0$.
9. **Leaderboard Formula:** weighted average of table- and column-level scores, equally weighted, in $[0, 1]$, higher is better.

$$\begin{aligned} \text{Leaderboard Score} &= 0.5 \times \text{Table Score} + 0.5 \times \text{Column Score} \\ \text{Table Score} &= \frac{P_T + R_T + F1_T}{3} \quad \text{Column Score} = \frac{P_C + R_C + F1_C}{3} \end{aligned}$$

5 What You Must Produce

5.1 Training Data Strategy

The released training split (301 examples) is your starting point. Decide how to compose, format, and (optionally) augment it. Augmentation is *not* required: investing your effort in schema serialization, output formatting, balancing, or post-processing instead of generating more data is an equally valid strategy. What matters is that your choices are deliberate and justified in the report. Examples:

- **Schema serialization:** full schema in the prompt (with or without types, PK/FK info), or a filtered subset? Different choices produce different sequence lengths and inductive biases.
- **Output format:** raw JSON, sorted tables/columns for stable decoding, an intermediate “reasoning” segment, etc.
- **Augmentation (optional):** paraphrase NL questions; synthesize additional NL-SQL pairs and label them with `sql_to_schema_links.py`; add “negative” examples; or use SNAILS

naturalness-modified identifier variants. Augmentation can help but can also inject noise—choose deliberately.

- **Curriculum / balancing:** weight under-represented databases (SBODemoUS sub-modules have only ~ 6 examples each), or multi-table-join examples.

You may also optionally define custom diagnostic metrics beyond the table/column P/R/F1 (e.g., a *join-coverage* score penalizing missed FK joins). Describe any such metrics and the failure modes they target in your report.

5.2 Automated Inference Script

Your `main.py` must run end-to-end via:

```
python3 main.py --input input_filename --output output_filename
```

Additional optional flags (`--batch_size`, `--checkpoint_path`, etc.) are allowed, but the above invocation alone must work. You may use `rapidfirerai`, `transformers`, `peft`, `vllm`, or any Python package; document non-standard installs in your README.

Input: JSON list of entries with:

- `question_id`: integer (1-indexed per file; *not* globally unique across files).
- `db_id`: string (one of 17; note four contain spaces, e.g. "SBODemoUS-Human Resources" maps to `schemas/SBODemoUS-Human_Resources.json`).
- `question`: string.

Output: JSON list of entries with:

- `question_id`: integer (must match an input `question_id`; output order doesn't need to match input).
- `schema_links`: object mapping table names to lists of column names. Tables referenced without specific columns (e.g., `select count(*)`) must appear with an empty list. Do NOT include `db_id` in the output. Match the casing in the schema files (matching is case-insensitive, but matching casing helps you debug).

Your `main.py` must locate schemas at `./schemas/` relative to its working directory—commit the released `schemas/` folder into your repo. TAs run from the repo root against a hidden `-input`.

Do NOT hardcode answers, hardcode absolute paths, or assume the validation set is present.

Inference time budget: predictions for ~ 100 hidden-test questions within **15 minutes** on the grading pod. Exceeding this terminates the run; partial output is graded.

5.3 Model Checkpoint and TA Workflow

What the TAs do. The TAs will *only run inference*. They will **not** re-train your model or replay your RapidFire experiments. Grading means: clone your repo on a fresh DSMLP pod (~ 24 GB GPU), install your declared dependencies, run `python3 main.py -input <hidden> -output <pred>`. Make that one command work, end-to-end, with no manual setup.

Packaging your trained model. Your final trained model artifact—the LoRA/QLoRA adapter, or the full fine-tuned weights if you did full FT—must live in **your private team git repo** (which the TAs and instructor already have access to). Do *not* push checkpoints, adapters, or any team artifact to a public HuggingFace Hub repo or any other public location; that would leak your work to other teams.

Your `main.py` should:

- Load the base model from HuggingFace Hub via the standard `transformers` API (e.g., `AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct")`). The base model is public and shared; no need to bundle it.
- **LoRA / QLoRA (recommended):** commit your adapter to a folder named `adapter/` at the repo root. Load it via PEFT: `PeftModel.from_pretrained(base, "./adapter")`. Adapters are typically 10–100 MB and commit cleanly to regular git. No checkpoint folder is needed—`main.py` loads the base from HF Hub and attaches your local adapter.
- **Full fine-tuning:** commit your full fine-tuned weights to a folder named `checkpoint/` at the repo root. Load it directly via `AutoModelForCausalLM.from_pretrained("./checkpoint")`. A full checkpoint for a 1.5B base is ~3 GB, which is too large for regular git; in that case provide a setup script that pulls the weights into `checkpoint/`, or use a **private** HuggingFace Hub repo with the TA accounts explicitly added (do not use a public HF repo).

State the chosen mechanism (LoRA adapter or full checkpoint, with how it gets to disk) in your README so the TAs know where the artifact comes from.

Out-of-band training and academic integrity. You are not required to do all training on DSMLP—the 30–60 GPU-hour DSMLP allocation is designed to be sufficient for an excellent submission, but if you have access to other compute, you may use it. Either way, your submission must honestly represent your own work:

- Your `logs/` directory must contain the actual RapidFire AI logs and metrics that produced the configs in your report—not fabricated or back-filled traces.
- Your git commit/push history must reflect genuine iterative development.
- Your submitted checkpoint must be consistent with the experimentation described in your report and logs.

Verification checks. To keep grading fair and honest, the TAs will spot-check every submission’s `logs/` directory against the report’s claimed configs and the git commit history. Most submissions will clear this without further action. If the spot-check surfaces anomalies—log/report/code inconsistencies, suspicious commit patterns, or a checkpoint that does not match the reported pipeline—your team will be called in for a brief F2F verification. You should be able to walk through your pipeline, explain your design decisions, and, if asked, launch a representative RapidFire AI training run live (on DSMLP or your own machine) from your committed scripts and configs. Keep your environment and at least one reproducible config in working order through the grading window in case you are called in.

Fabricating logs, plagiarizing checkpoints, or claiming experimentation you did not perform is an academic integrity violation.

5.4 Project Report

A succinct PDF report (say, 4 pages) with five sections:

- a. **Training Data Methodology:** how you composed your training data, serialization and output format, any augmentation, and why. Any diagnostic metrics you defined.
- b. **Final Pipeline Architecture:** base model, adapter (LoRA/QLoRA config), prompt template, output decoding, and post-processing (e.g., JSON repair, identifier validation).
- c. **Experimentation Methodology:** which config combinations you compared via RapidFire AI, why those knob values, and how you used the multi-config API. Demonstrate at least 8 meaningfully distinct configs. Include a table of all config knobs.
- d. **Results and Analysis:** tabulate/plot all configs you tried and how you arrived at the final pipeline. What worked, what didn't, why. Which knobs mattered most for table- vs. column-level metrics? Surprises? What you'd try with more time/budget?
- e. **AI Tools Statement:** brief description of how you used AI assistants (Cursor, Claude Code, etc.).

5.5 Git Repository

Set up a private git repo within 24 hours of release; add all TAs and the instructor as collaborators (GitHub IDs on Piazza). Reusing your Project 1 repo is fine if your team is unchanged.

Your repo must contain: all code and configuration; training-data manifests; the `schemas/` folder; validation-set output files; `main.py`; `README.md`; the project report PDF; a `logs/` subfolder with all RapidFire AI experiment logs and metrics (`rapidfire.log` and metrics from your `experiments_path`); and your trained-model artifact—either an `adapter/` folder (LoRA/QLoRA) or a `checkpoint/` folder (full fine-tuning), per Section 5.3. Push regularly—this helps the TAs verify progress and assists with partial-credit debugging.

Important: Your git history must reflect genuine iterative development. Do NOT dump all code from an AI tool in one commit. Forging history is an integrity violation. For your trained model artifact, see Section 5.3.

See the course Projects overview page for full policies on teams, integrity, and git.

6 Grading Your Submissions

6.1 Auto-Grading Your Code: 10%

Your `main.py` is auto-graded on both the released validation set and our hidden test set. Validation output you submit is cross-checked against your code's output. Your project score and leaderboard metrics are based only on the hidden test set. Code that fails to run, produces inconsistent output, or scores much worse than expected will lose points.

Score expectations: a well-executed sub-2B SFT pipeline should achieve a leaderboard score of at least ~ 0.50 on the hidden test set, and competent teams should reach $0.60+$. Scores significantly below 0.50 typically indicate a pipeline issue (broken JSON parsing, casing mismatches, dropped empty-list tables, output truncation) rather than a model limitation—check your validation-set output with `eval.py` carefully and fix obvious bugs before submitting. These numbers are calibration anchors, not hard cutoffs.

6.2 Training Data and Methodology: 4%

TAs inspect your training-data preprocessing scripts, any augmented data you generated, and the methodology section of your report. Grading is on rigor and justification, not volume of augmented data. Trivial or undocumented augmentations, blatant duplication, or unjustified design choices will lose points.

6.3 Project Report Grading: 6%

TAs grade the report using both LLMs and manual reading. Evaluation is on depth and rigor of experimental methodology, evidence of systematic experimentation (not ad-hoc trial and error), and clarity of reasoning over results. With no F2F interview on this project, the report is the primary place where you demonstrate that you understand your own pipeline and experimentation—write it accordingly.

6.4 Extra Credit based on Leaderboard

The leaderboard is NOT part of the base grade; it exists to motivate excellence beyond expectations. Top 10th percentile of teams: 2% extra credit toward the course total. Next 10th percentile: 1%. Your leaderboard position uses your final submission on the hidden test set.

7 Submission Logistics

Submit via Canvas. Only one submission per team; multiple resubmissions keep only the final one. Submit a single zipped folder `TeamID.zip` containing:

1. Any custom training data files you generated, as JSON (only if applicable).
2. Output JSON for the released validation set from your final pipeline.
3. Zipped folder of your final git repo. Must include `main.py`, `README.md`, `schemas/`, `logs/`, and your trained-model artifact (`adapter/` for LoRA/QLoRA, or `checkpoint/` for full FT) at the top level.
4. Project report PDF.

Also add all TAs and the instructor as collaborators on your private git repo within 24 hours of release.

Experimentation vs. Final Pipeline. Your experimentation can be interactive, documented via logs and the report. Your final submission must be a deterministic script that runs inference using your chosen fine-tuned model—TAs will NOT re-train or replay experiments. See Section 5.3 for packaging.

8 Timeline

Milestone	Date
Release Date	Mon, May 18
Discussion by TA	Wed, May 20
Git Repo Deadline	Tue, May 19, 11:59pm PT
Due Date	Mon, Jun 1, 11:59pm PT

9 Tips

- **Start with the tutorial notebooks.** Get RapidFire AI's SFT path working end-to-end first. A working baseline—small model, LoRA, raw-JSON output, single epoch—should be doable in a day. Add sophistication incrementally. “Premature optimization is the root of all evil!”—Knuth.
- **Budget your GPU time.** Downsample data and use a tiny base while you stabilize the pipeline; scale up only after tokenization, prompt template, parsing, and eval-loop wiring all work. Idle runs on a large base with a buggy prompt template are how budgets vanish overnight.
- **Validate output format aggressively.** Output must parse as JSON conforming to the schema-link contract. Add JSON-repair / schema-validation post-processing; budget for truncated or malformed JSON.
- **Beware data leakage.** SNAILS gold queries are public on GitHub, but the hidden test set (Section 4) includes questions we authored over the same schemas. Memorizing from a public copy is cheating *and* unlikely to help. Build for generalization, not the leaderboard alone.
- **Use RapidFire AI's multi-config framework intentionally.** Define config groups with `RFGGridSearch` or `RFRandomSearch`. Use `get_results` and `get_runs_info` as needed. Use Interactive Control Operations (Stop, Resume, Clone-Modify) to redirect compute from unpromising configs.
- **Watch for hallucinations.** Identifiers not in the target schema count as false positives, lowering precision. Run `eval.py's --per_question_out` mode to see where hallucinations happen, and consider a post-processing pass that drops identifiers absent from the input schema.
- **The released training data is small (301 examples across 17 schemas).** A baseline can train on it as-is. Whether to grow it—via paraphrases, LLM-generated NL-SQL pairs labeled by `sql_to_schema_links.py`, or SNAILS naturalness variants—is your call. Prompt template, output format, and post-processing often matter more than raw example count. Document whatever you choose.
- **Think about the column-level metric.** Column F1 is harder than table F1 because the same column name can exist across many tables. Your prompt template and output format materially affect this.
- **Training data and report are graded deliverables, not afterthoughts.** Good training data design is hard—invest in it and it will also help you debug.
- **Collective team responsibility.** Both members of a 2-person team are responsible for the full submission and must know each other's work. By default both members get the same score.

References

- [1] K. Luoma and A. Kumar. “SNAILS: Schema Naming Assessments for Improved LLM-Based SQL Inference.” *Proc. ACM SIGMOD*, 2025. Artifacts: <https://github.com/KyleLuoma/SNAILS>; documentation: <https://snails.jp8.dev>.